

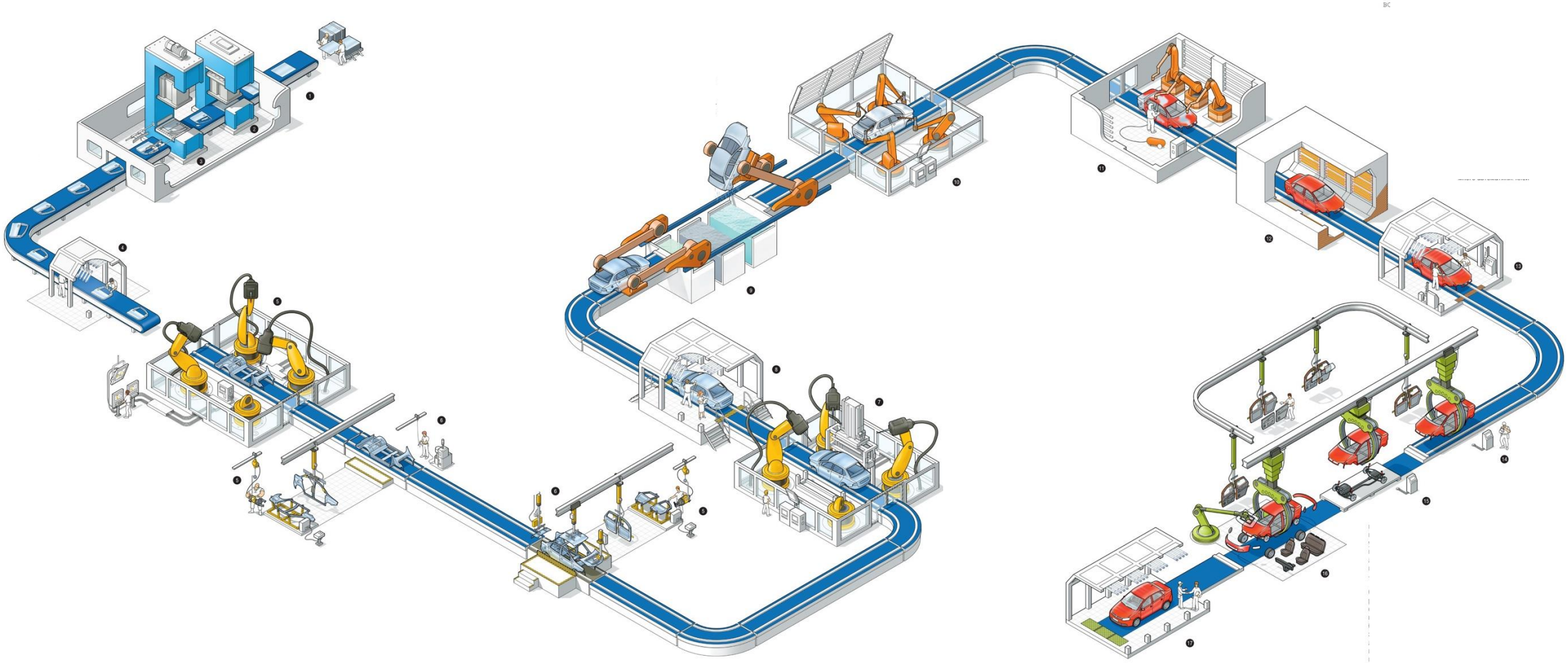
ASD02 Project Management 1

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

29 July 2025

Project vs process



Project vs process

	Process	Project
Nature	routine	ad hoc
Duration	continuous	fixed start + end
Inputs/outputs/tasks	known/doc'ed	uncertain
Success	>99%	~50%
Emphasis	Small improvements	creation from scratch
Control	fixed measurements	outcome vs desires
Measurement	$ input = $ outputs	meets requirements

Process good, project bad.



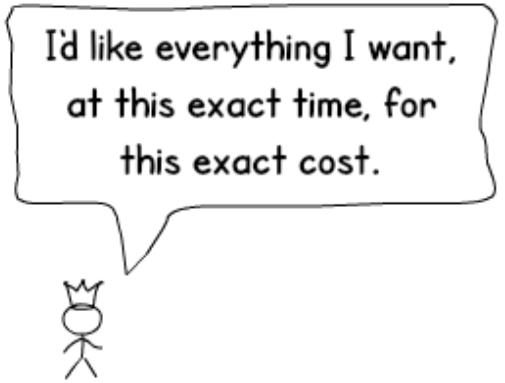
CSC3003S ASD03
Project Management 1

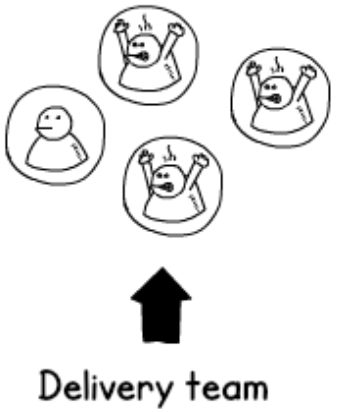
A. Overview

B. Planning and Scope

C. People

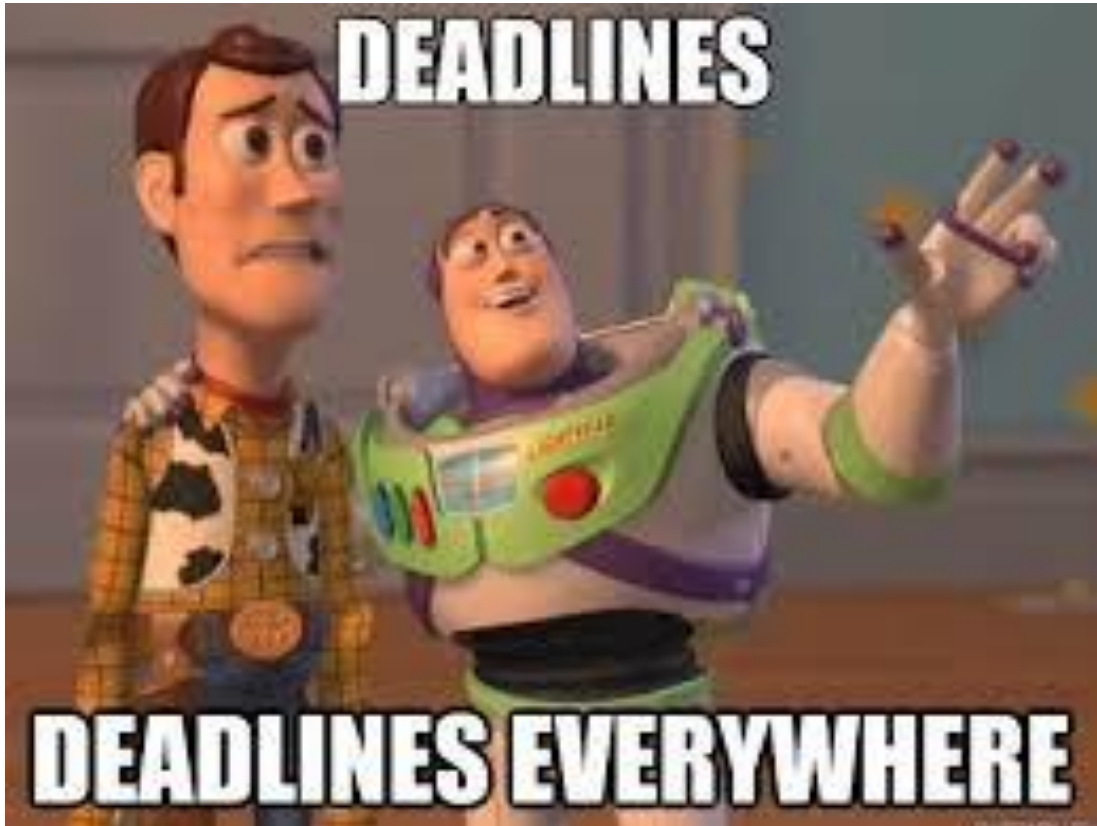
D. Metrics





Project Management

- Ensuring software is delivered **on time, within budget,** and **as per requirements (on quality).**
- Software development is always subject to **budget and time constraints.**



Project Management

- Assigned Reading: Bennett Chapter 22 (online)

  Managing Object-Oriented Projects (online) (on Amathuba)

- Systems development is complex and requires careful project management.
  - Why? ***dependencies, people and organising***
- Inter-dependencies between the artefacts of software development
  - production has to be planned, monitored, and co-ordinated
    - To make development efficient, effective, and on time.
- Often involves many developers, some with specialized skills, needed at different times.
- Activities must follow a particular sequence.
  - E.g. can only test when have developed the components being tested!
  - test scripts and test harnesses may be prepared earlier in the project.

Brooks: No silver bullet …

Development difficult because [1]

- essential problems
  - due to requirements, scope
- accidental problems
  - due to process, tools, people
- Complexity *is* the problem
  - must manage complexity
  - only CS spans ~10^{13} orders of magnitude

"That is, *there is no single technique that fixes the inherent complexity of software design and development*"

Ineffective designs due to:

- complex soln to simple problem
- simple, faulty soln to complex problem
- complex, faulty soln to complex problem

[1] Brooks, F.P., "No Silver Bullet: Essence and Accidents of Software Engineering", 1987

Dijkstra: Manage complexity

- Minimize *essential* complexity in brain at any one time
  - problem decomposition: break project into simple pieces
  - loose coupling
  - clean interfaces (no hidden gotchas)
- Keep accidental complexity from proliferating
  - tools (language)
  - framework
- *All other goals are secondary* to managing complexity

Good design properties

- Minimal complexity: *easy to understand*
- Easy of maintenance: *documented, understandable*
- Loose coupling: *interfaces: abstraction/encapsulation/info hiding*
- Extensible: *easy to extend*
- Reusable: *DRY - don't repeat yourself*
- High fan-in: *reuse low level classes often*
- low fan-out: *high level class/function uses few lower-level ones*
- Leanness: *"Make everything as simple as possible, but not simpler"*
- Stratified: *consistent view at any level of decomposition*
- Standardized: *use common, proven patterns*

Project Management Tools

Choose one or mix and match for your capstone project!











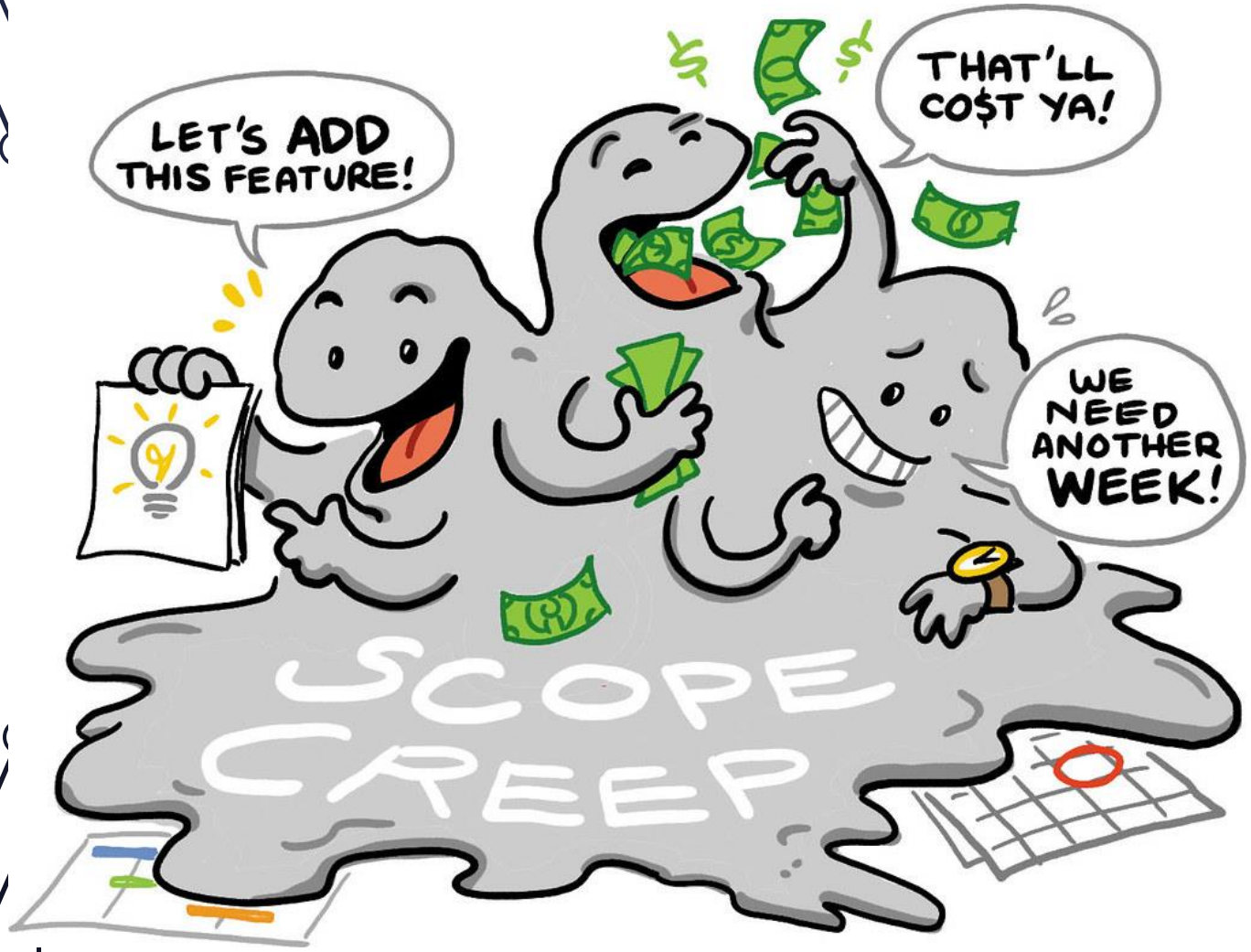








and more…



CSC3003S ASD03 Project Management 1

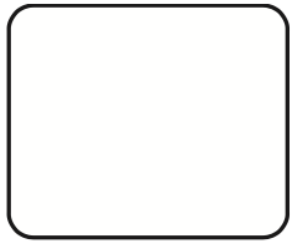
A. Overview

B. Planning and Scope

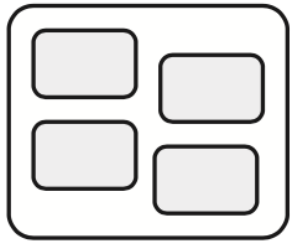
C. People

D. Metrics

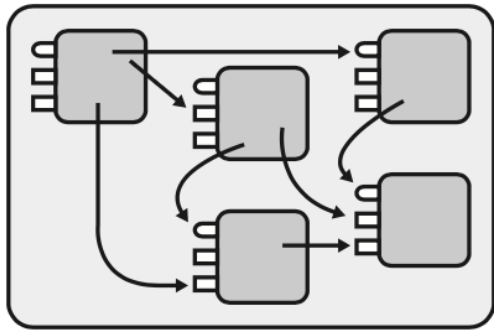
Design levels



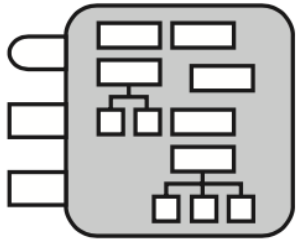
Project



Subsystems



Classes/packages



Methods + data in classes

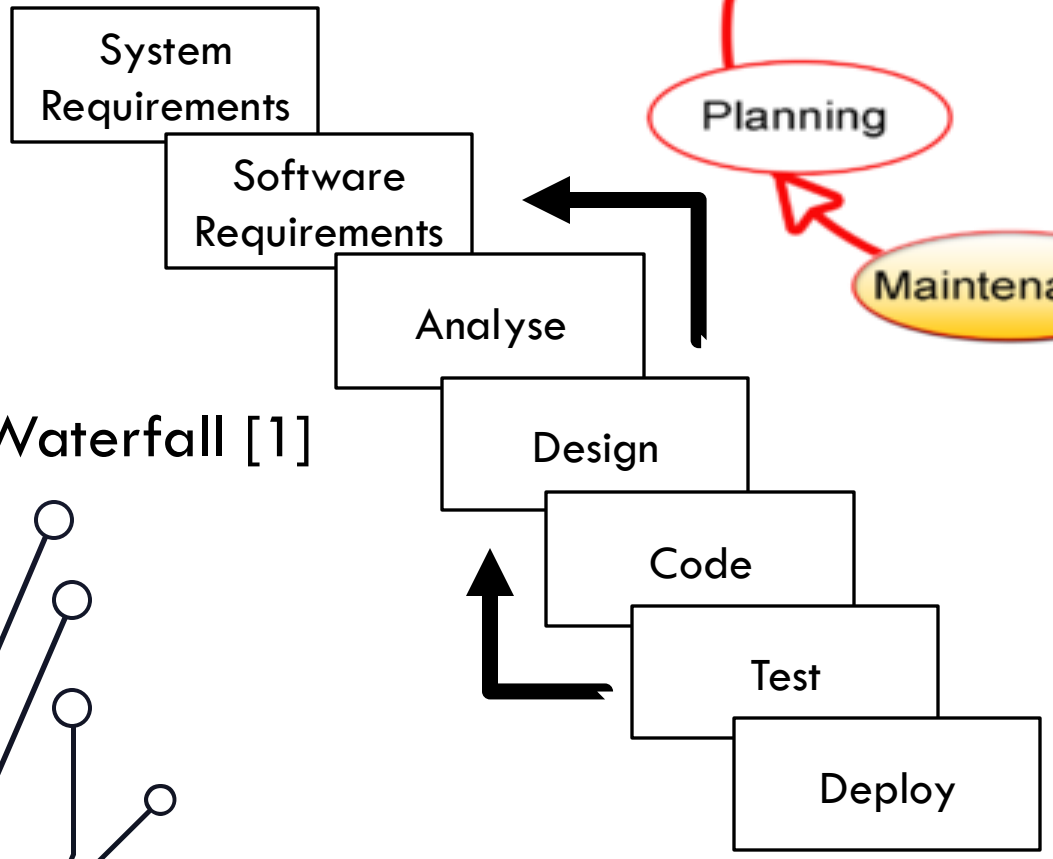


Source code

Design level 0: Pick methodology

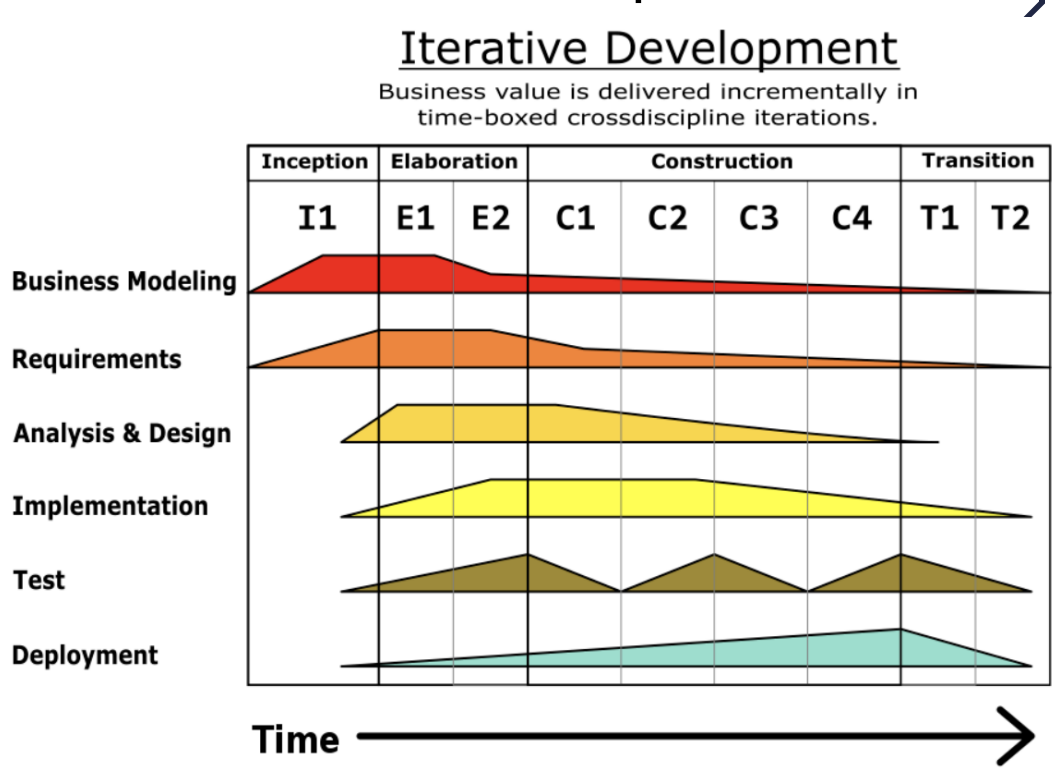
SDLC: Software development life cycle





Waterfall [1]

Unified process





[1] Royce, Winston, Managing the Development of Large Software Systems, 1970

Design level 1: Project



Requirements

- Functions and features that are to be delivered to end-users
- Data that are input and output
- Content that is presented to users as a consequence of using the software
- the performance, constraints, external interfaces and reliability that bound the system

Defining Scope

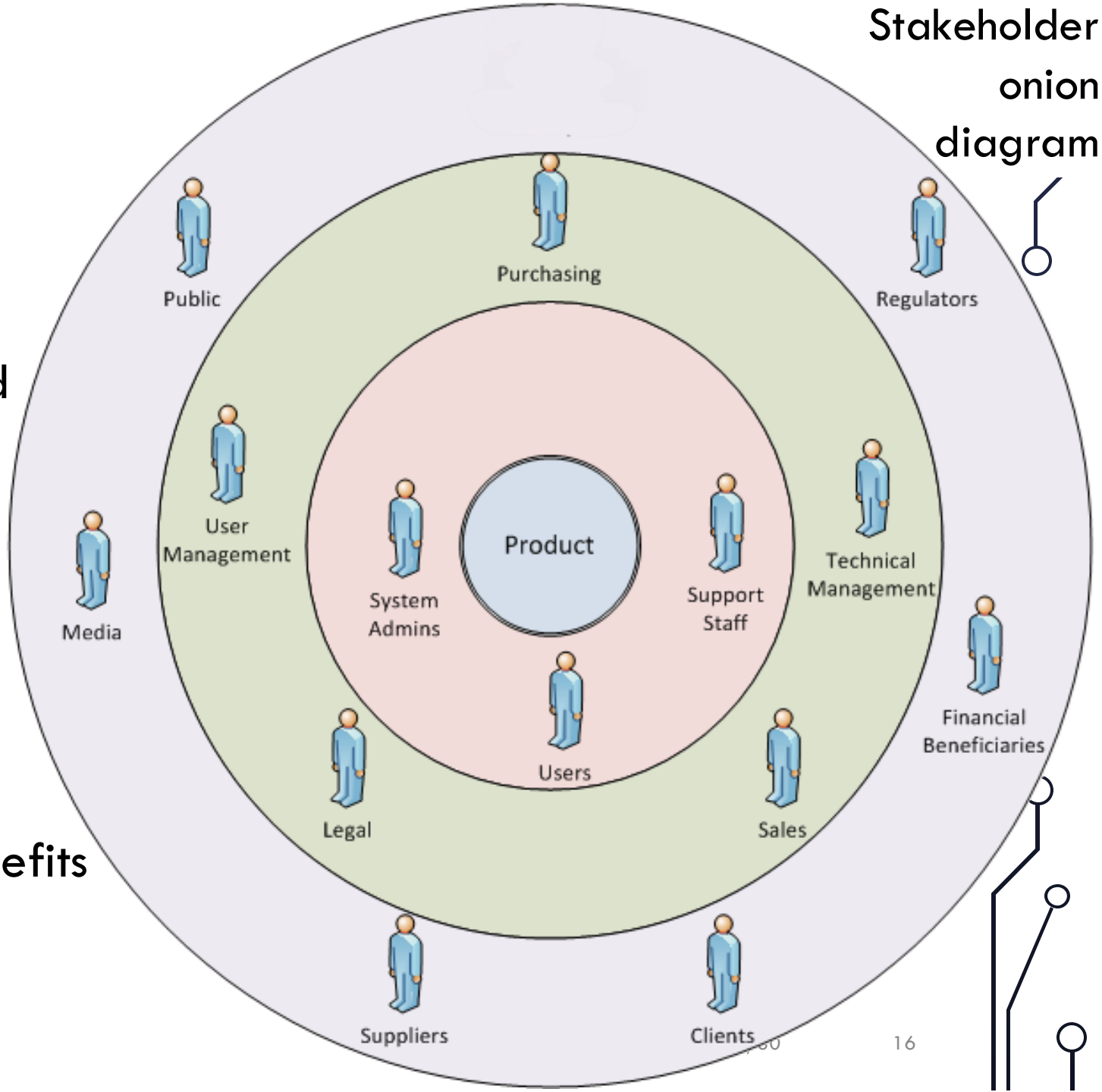
- Scope defined by either:
  - Narrative descriptions

    after communication with stakeholders.
  - Use-cases
    - scenario-based description of the user's interaction with the software from the user's point of view.
- Requirements
  - Functionality
  - Performance
  - ....
- Constraints: limits
  - external hardware, memory, existing infrastructure
  - time/budget

Project Planning I
Goals — aka Scope

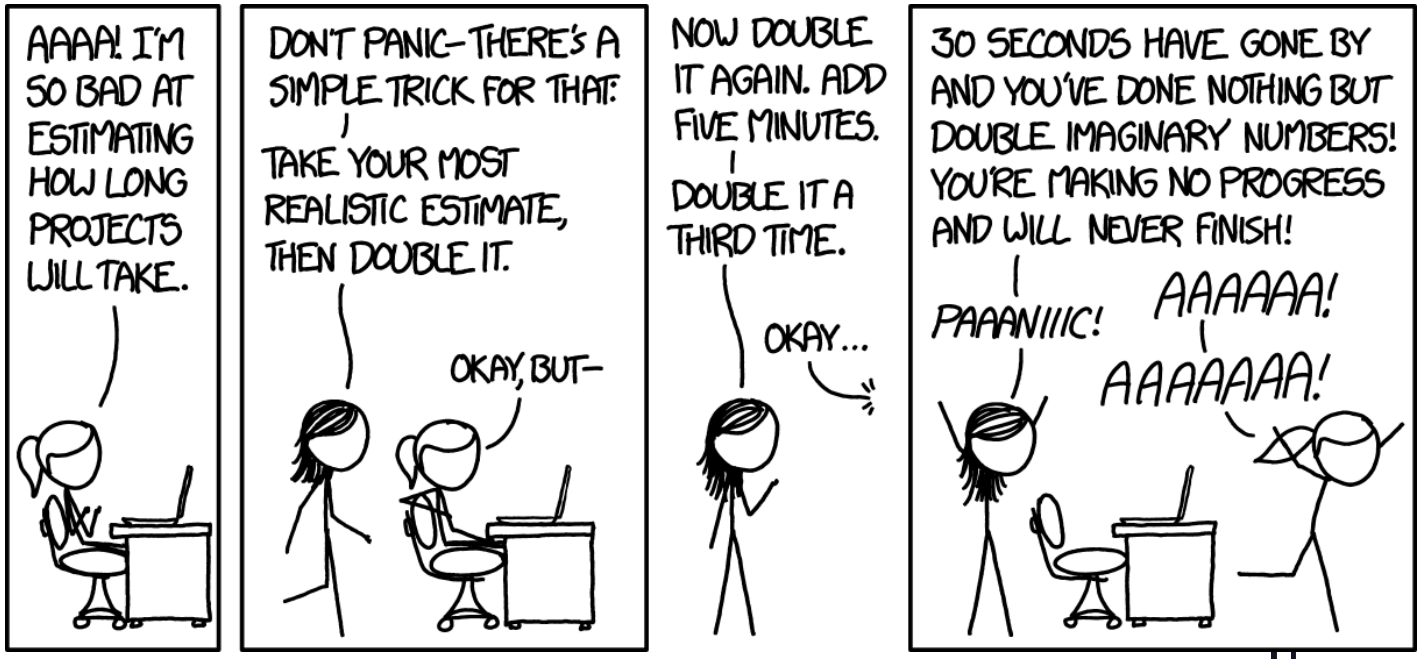
- Goals derive from the needs and expectations of stakeholders:
  - Stakeholder = someone impacted by the project
    - project sponsor
    - client for the deliverables
    - users of the system
    - project manager and team
- Find true needs creating real benefits
  - Prioritize these

Stakeholder onion diagram



Determining Feasibility

- Given the scope, determine the resources required.
  - Not just $$$
- Make sure you have an answer to the question:
  - Do we have the resources we need to build this software?
- Resources are:
  - people
  - hardware and tools
  - reusable components



Level 1 Requirements...

Why prepare?

- Reduce risk
  - More preparation => greater chance of success
- Efficiency
  - 80% of development time is spend finding and fixing bugs [1]

Table 3-1 Average Cost of Fixing Defects Based on When They're Introduced and Detected

	Time Detected				
Time Introduced	**Requirements**	**Architecture**	**Construction**	**System Test**	**Post-Release**
Requirements	1	3	5–10	10	10–100
Architecture	—	1	10	15	25–100
Construction	—	—	1	10	10–25

[1] McConnel, Code complete 2nd edition, 2004

Level 1 ...Requirements

Why **explicit** requirements?

- Allow client (not programmer) to drive functionality
- Explicit requirements can be reviewed by client, code cannot
- Changing requirements later is more expensive
- Specifying requirements is key to project success
  - Stable requirements are the holy grail => no cost overruns
  - Make sure users know cost of requirement changes
  - Use development approach that allows for change
    - use change control procedure
    - e.g. Agile/evolutionary delivery

McConnel, Code complete 2nd edition, 2004

Requirement checklist, have you specified (examples)...

Functional

- All inputs
- All outputs
- File formats
- Hardware + software interfaces
- External interfaces (incl. errors/protocols etc.)
- All user tasks
- User interface
- data in-to and out-of user tasks

Quality (non-functional)

- Response time (latency)
- Performance
- Security
- Reliability
- Resource use (e.g.: memory/storage)
- Maintainability
- Definition of success/failure (benchmark)

(see ISO 25010)

...Requirement checklist, are requirements (examples)

Complete

- Has missing info at start been documented?
- If program satisfies req. will it be accepted?
- Are req. realistic/relevant
  - removed all impossible req.
  - removed all nonsense

Correct

- Written in users' language? users agree?
- No conflicting req.
- Acceptable tradeoffs for competing req. e.g.: *speed/quality*
- *Detailed enough, consistently detailed*
- *Understandable to coders? coders agree?*
- *relevant*
- *testable*
- *Adaptable (possible changes spec'ed)*

Architecture

Programming languages/tools

Program organisation – subsystems

Major classes

Data design

Business rules

User interface

Resources (database/memory/threads)

Security

Performance

Scalability

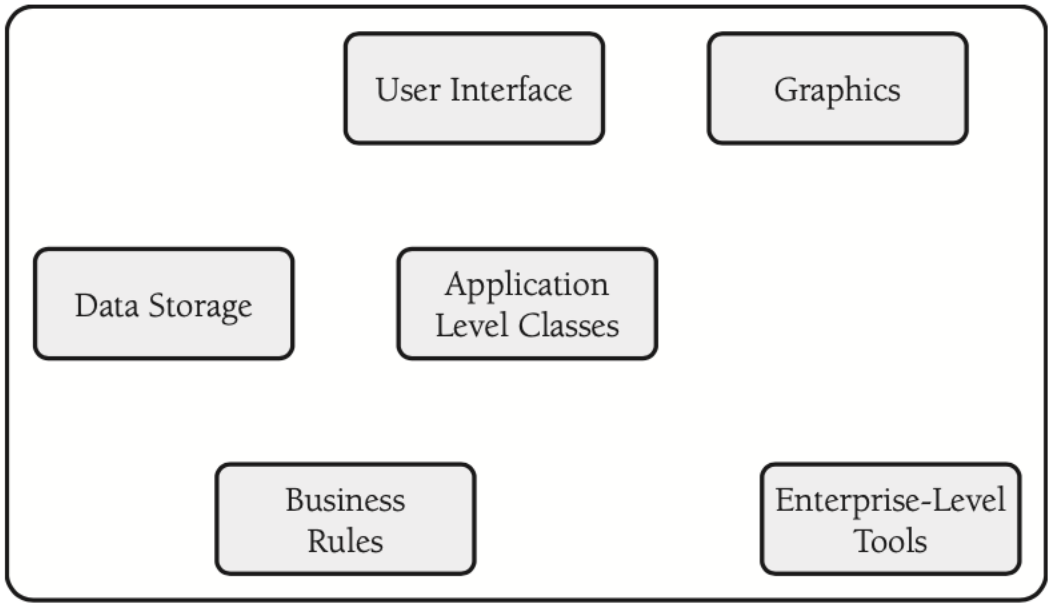
I/O

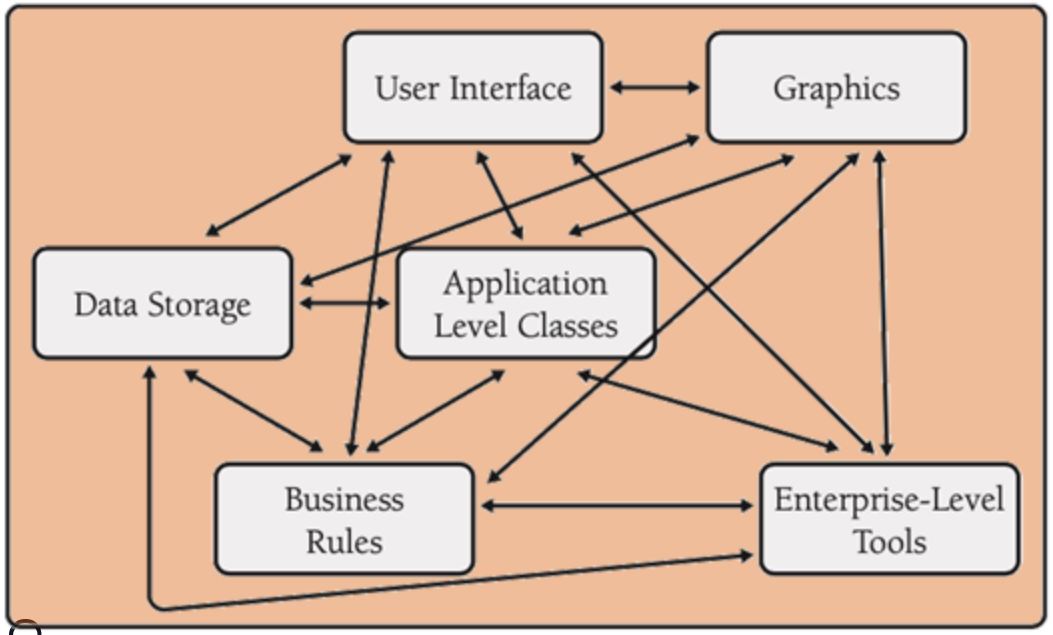
Error processing/Fault tolerance (system wide implications)

Buy-vs-build (frameworks)

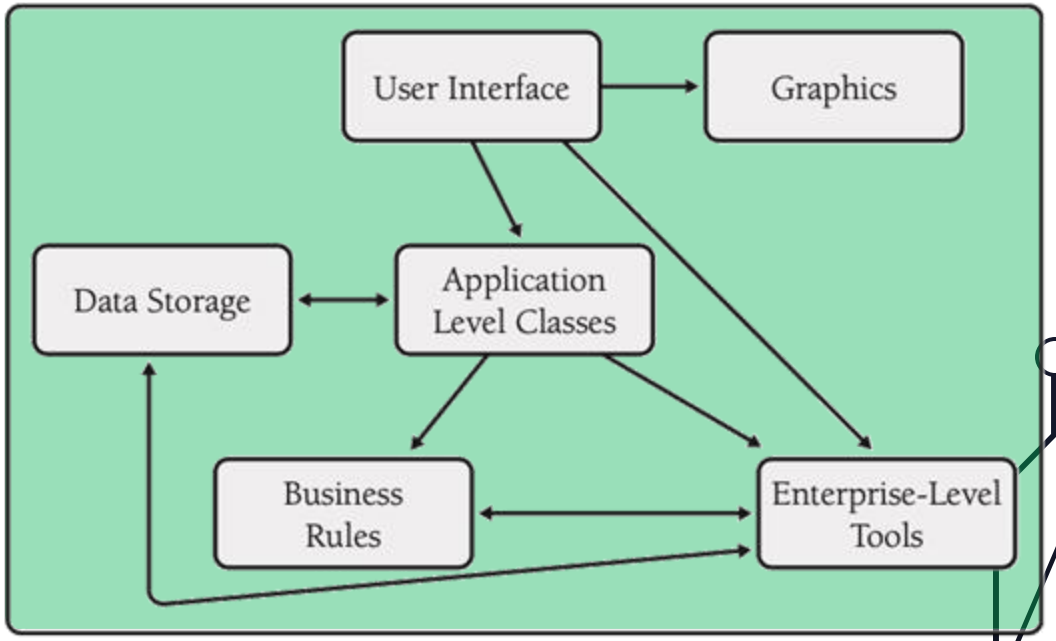
Design level 2 Subsystems

- Plan subsystems
  - Self contained units
  - Assign to different people
- Pick frameworks ('buy vs build')
- Plan interactions
  - based on hierarchy



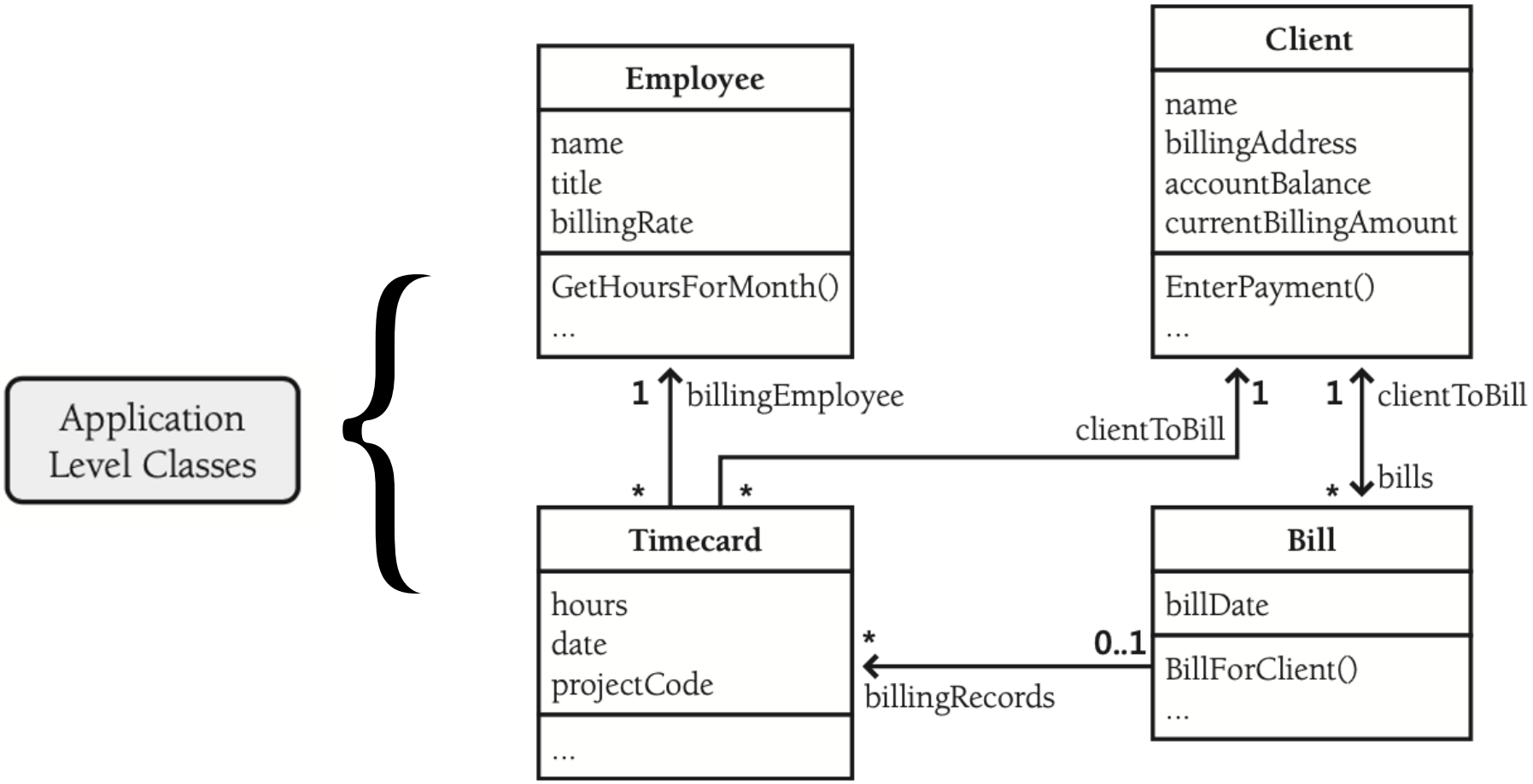


Complex: no limitations on interactions, high fanout



Simple: rules-based interactions, low fanout

Design level 3: Division into classes



Design level 4: Division classes into routines

Steps: iterative, *can be done out of order*

- Design object attributes
  - methods
  - data
- What can be done to each object
- Interactions with other objects
- Partition into Private/Protected/Public parts
- Define public interface

Depends on programming paradigm:

- *OOP: - Real world objects*
- *DOD: - Program data flows*

Design level 5: Design routines + data

- Check requirements (e.g.: performance, functionality)
- Give it a good name: verbObject (name *documents behaviour*)
  - use naming convention
- Add needed documentation, *esp. requirements*
- Follow design patterns (*depends on language*)
- Design parameters/return values (*const correctness*)
- Design invariants/checks (assertions/exceptions)
  - defend against bad input
  - pre-conditions (inputs/object start state)
  - post-conditions (outputs/object end state)
  - assertions
- Implementation (*comes later in some methodologies*)
  - start with pseudo code

Project Planning I
SMART Goals

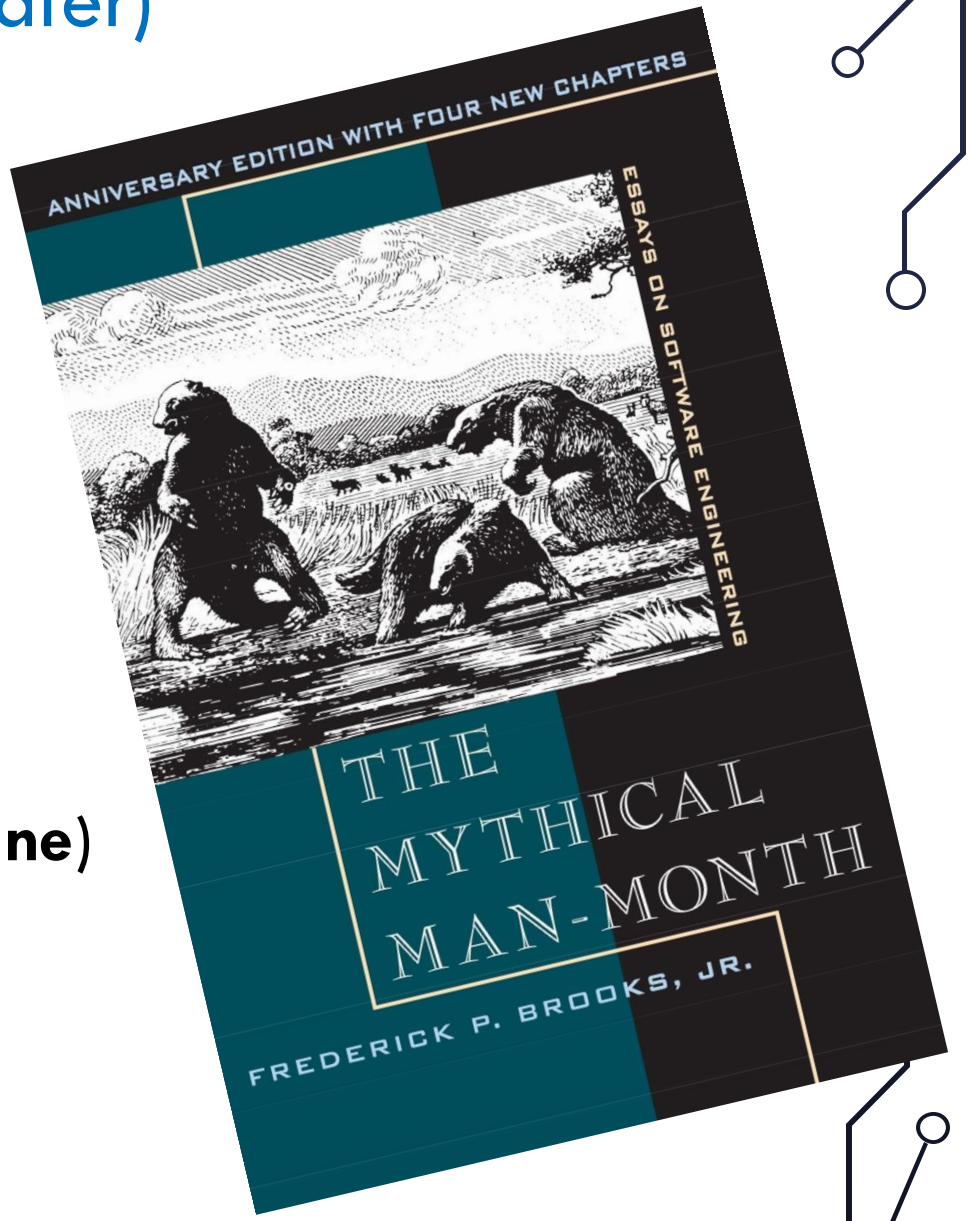


Project Planning II
Deliverables

- Deliverables = items that have to be delivered to meet the goals
  - Verifiable and specific
  - E.g.: report/executable code, etc.
  - Has a stakeholder who needs it.
- Say when it is to be delivered
- Give quality standards, for example:
  - software runs!
  - document meets specified format

Project Planning III Schedule (+more later)

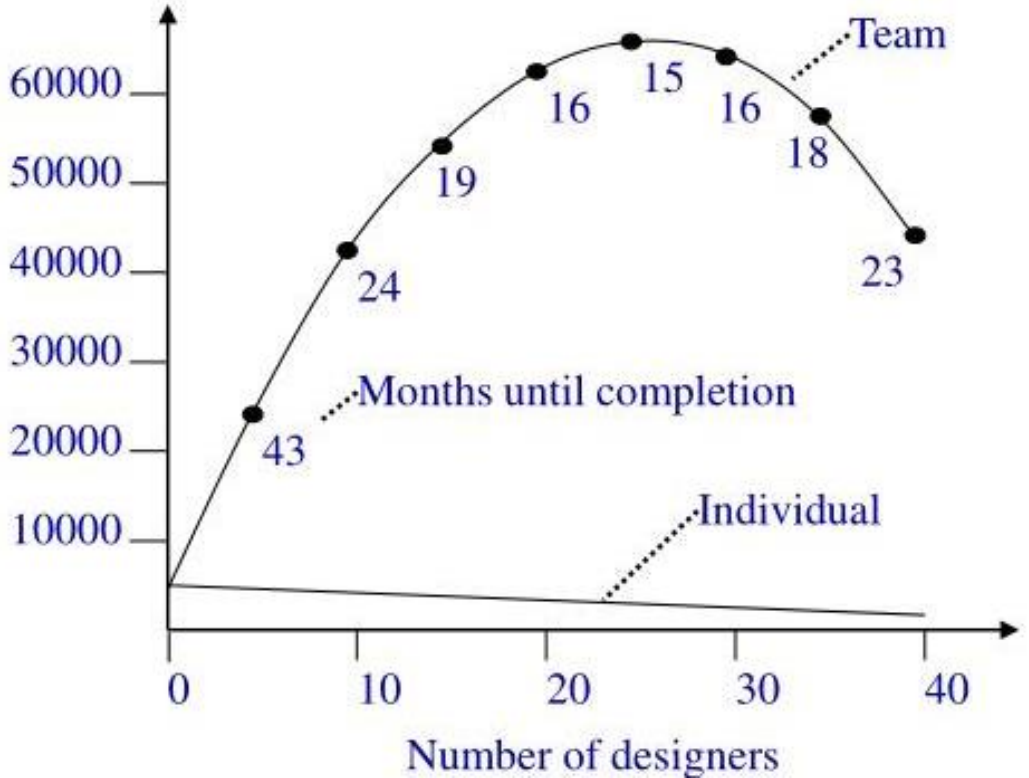
- The time (**effort)** required
- The **people** + other resources required
- Update deliverables => schedule
- If the schedule is unrealistic, either:
  - delay deadline
  - additional resources
    – more expensive, mythical man-month!
  - reduce scope – fewer deliverables
    (**Only thing that makes sense for Capstone**)



The Mythical Man-month

- In theory, adding designers to team reduces project completion time
- In reality, productivity per designer decreases due to complexities of team management and communication
- In the software community, known as "the mythical man-month" (Brooks 1975)
- At some point, can actually lengthen project completion time! ("Too many cooks")

- 1M transistors, 1 designer=5000 trans/month
- Each additional designer reduces for 100 trans/month
- So 2 designers produce 4900 trans/month each



Project Planning IV
Supporting Plans

Human Resource Plan

- Plan key staff & organisations: describe: skills, roles, and responsibilities
- Describe no. and type of people needed
  - start dates, estimated duration and how to them

Communications Plan

- Who needs to be updated about the project?
- How to inform?
  - weekly review meeting
  - progress reports/demos
  - revised schedule

Risk Management Plan — see later

- Identify as many risks as possible
  - ARTA: Accept/Reduce/Transfer/Avoid



Management Activities

People: Project Managers, Team Leaders, Software Team, Customers, End Users

Product Scope, requirements, decomposition

Process Start with a skateboard, iteratively refine

- Software Development Life Cycle
- Unified Process:

Project Resources, scheduling, risks, tracking.

Price Budget, controlling cost, bookkeeping



CSC3003S ASD03
Project Management 1

A. Overview

B. Planning and Scope

C. People

D. Metrics

Roles and Responsibilities

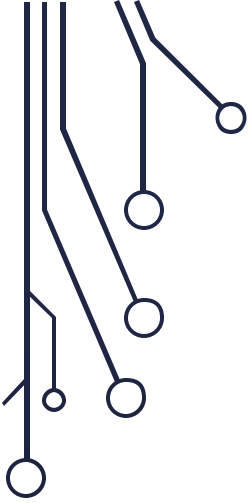
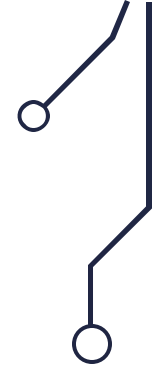
- A team needs people with *different* skills
- Range of duties even in a small project
  - Project management: Strategist, Leader, Politician, Project Facilitator, Administrator
  - Systems analysis: Stakeholder needs, Interaction Designer, Cost estimator
  - User interface designer: user stories
  - Architect: Application overview, performance
    - Middleware – software layer between the operating system and the applications on a distributed computer network
  - Specialists as needed: database, games engine, mobile development, …
  - Documentation

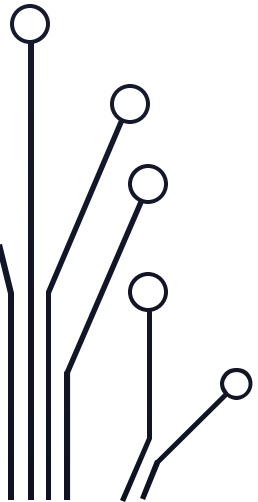
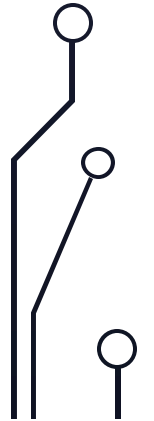
Choosing People

- People you know and have worked with before
- Information from candidates about their background and experience (CV)
  - Best evidence to judge suitability
- Information from interviewing candidates
  - Mainly judge communication and social skills
  - Subjective judgements thus *not* reliable
  - Can also ask to perform specific exercises
- References and recommendations from people who have worked with them
  - Effective when you *can rely on* the people making the recommendation.

Management and Team Success

- Most software engineering is a *group* activity
  - Non-trivial software projects **cannot** be done by one person
  - People motivated by success of the group *and* their own personal goals
- Individual success depends on:
  - Ability and interest for work at hand
  - Experience and training with similar applications, development tools, programming languages
- Team success depends on:
  - **Ability to communicate** and express ideas in the team
    - be heard or be herded
  - **Group interaction** is a key determinant of group performance.
- Management skills
  - Limited flexibility in group composition: do the best with people available



THE AUTHOR OF THE WINDOWS FILE COPY DIALOG VISITS SOME FRIENDS.

CSC3003S ASD03
Project Management 1

A. Overview

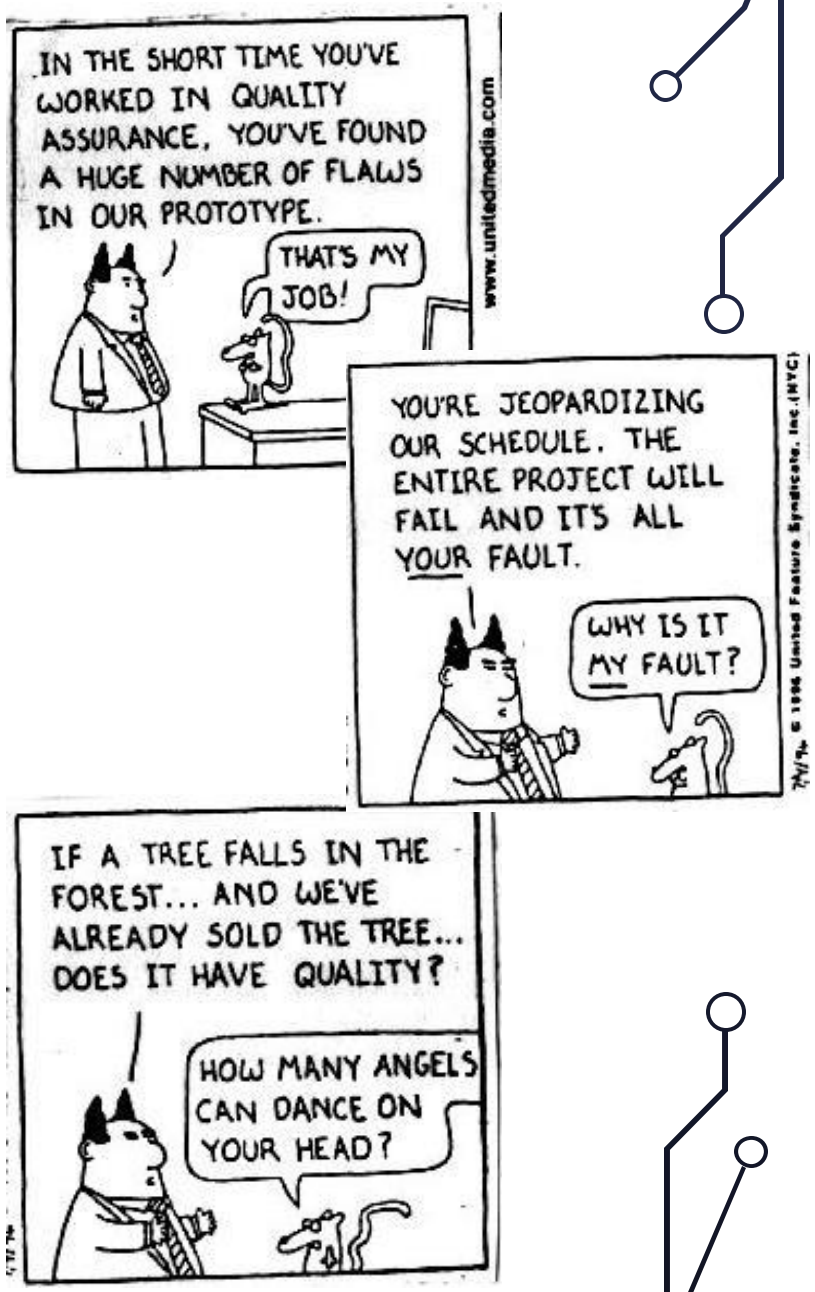
B. Planning and Scope

C. People

D. Metrics

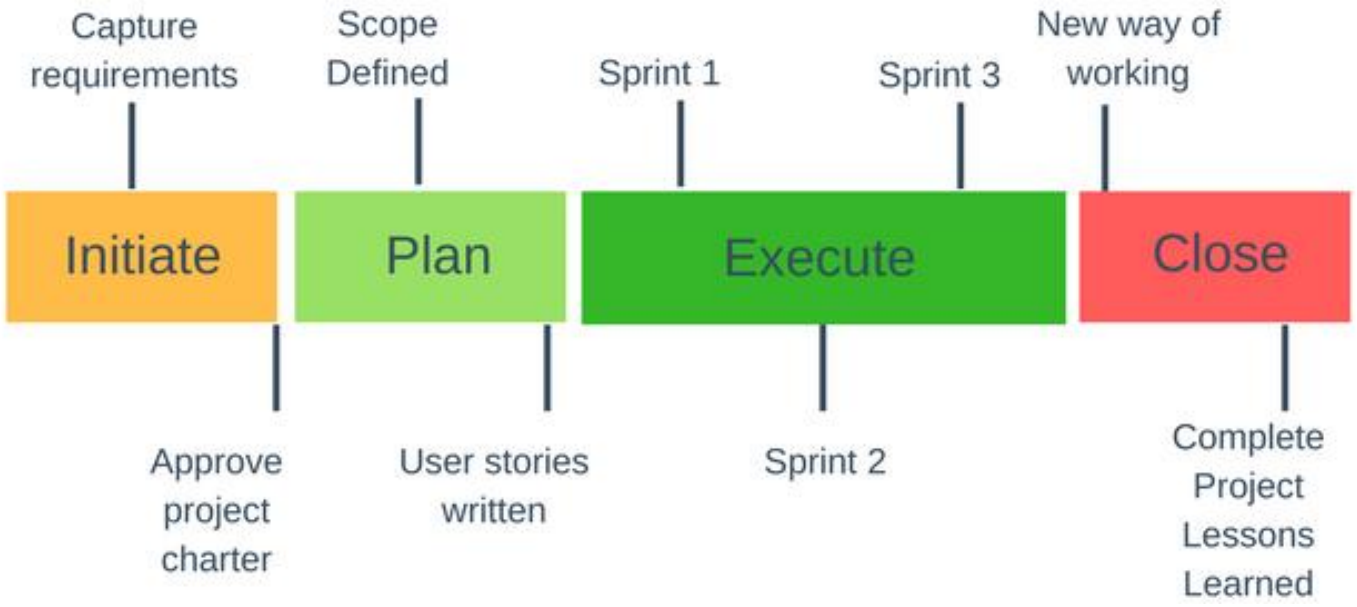
Milestones and Deliverables

- **Activity:** Task that takes time
  - Duration
  - Due date
  - Precursor: activity preceding others that depend on it
- **Milestone:**
  - Completion of an activity
  - Recognisable end-product of a task
    - hand over system for testing
  - Requires a formal, measurable output
    - "Coding 80% complete" is not adequate
- **Deliverable:**
  - A project result that is delivered (to customer)



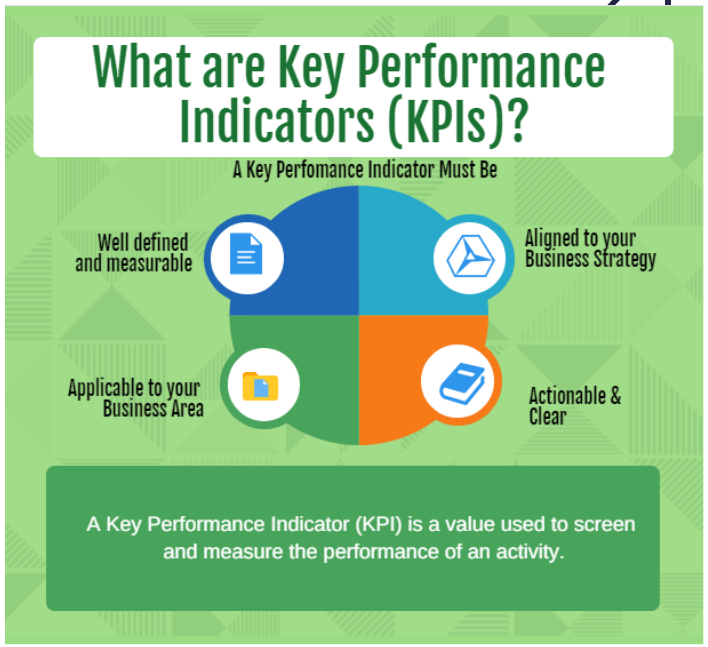
Put simply. Deliverable vs Milestone

- **Deliverables** = *measurable*, *tangible* outcomes of the project.
  - Releases, Documentation, Reports. Developed in alignment with the goals of the project.
- **Milestones** = *checkpoints* throughout the life of the project.
  - Identify when one or multiple groups of activities have been completed → a notable point has been reached in the project.

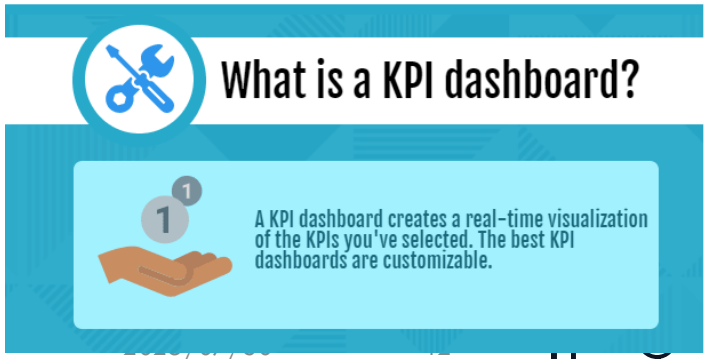


Why Software Metrics?

- To plan and manage a software development project
  - Need to estimate the resources required for each of its constituent activities.
    - subjective perceptions of the activity
    - based upon measurements of size and complexity
      - activity itself
      - artefact that is produced.
  - Software metric measures some aspect of software development
    - project level—cost/duration
    - application level —size/complexity.







Remember: Metrics are Smart!



SMART Metrics

- Counts
- Percentages
- Sums or Totals
- Averages
- Ratios

KPI GROUP	METRICS	TARGETS PER MONTH
CODE QUALITY	Maintainability index	Between 20 and 100: higher values are better
	Cyclomatic complexity	1 < target < 10
	Depth of inheritance	0
	Class coupling	0
	Lines of code	Depends on an app's complexity but the lower the better
TESTING QUALITY	Test coverage	80% or more + track the general trend
	Defects found in UAT	30 or less + track the general trend
	Defects found in production	No more than 10% out of all bugs + track the general trend
SOLUTION AVAILABILITY	MTBF	Track the general trend, aim to increase numbers
	MTTR	Track the general trend, aim to decrease numbers
	Page load time	Less than 2 seconds
SOLUTION SECURITY	Number of unavailability cases	1-5 (including deployment unavailability)
	Number of vulnerabilities found by regular penetration testing	0-3 + track the general trend
	Number and severity of security incidents	0-1 for High severity incidents and 0-2 for Medium and Low severity incidents
USER SATISFACTION (a survey with rankings from 1 (completely not satisfied) to 5 (completely satisfied)	Meeting user expectations on functionality	Not less than 5
	User interface convenience	Not less than 5
	Stability of software performance	Not less than 4

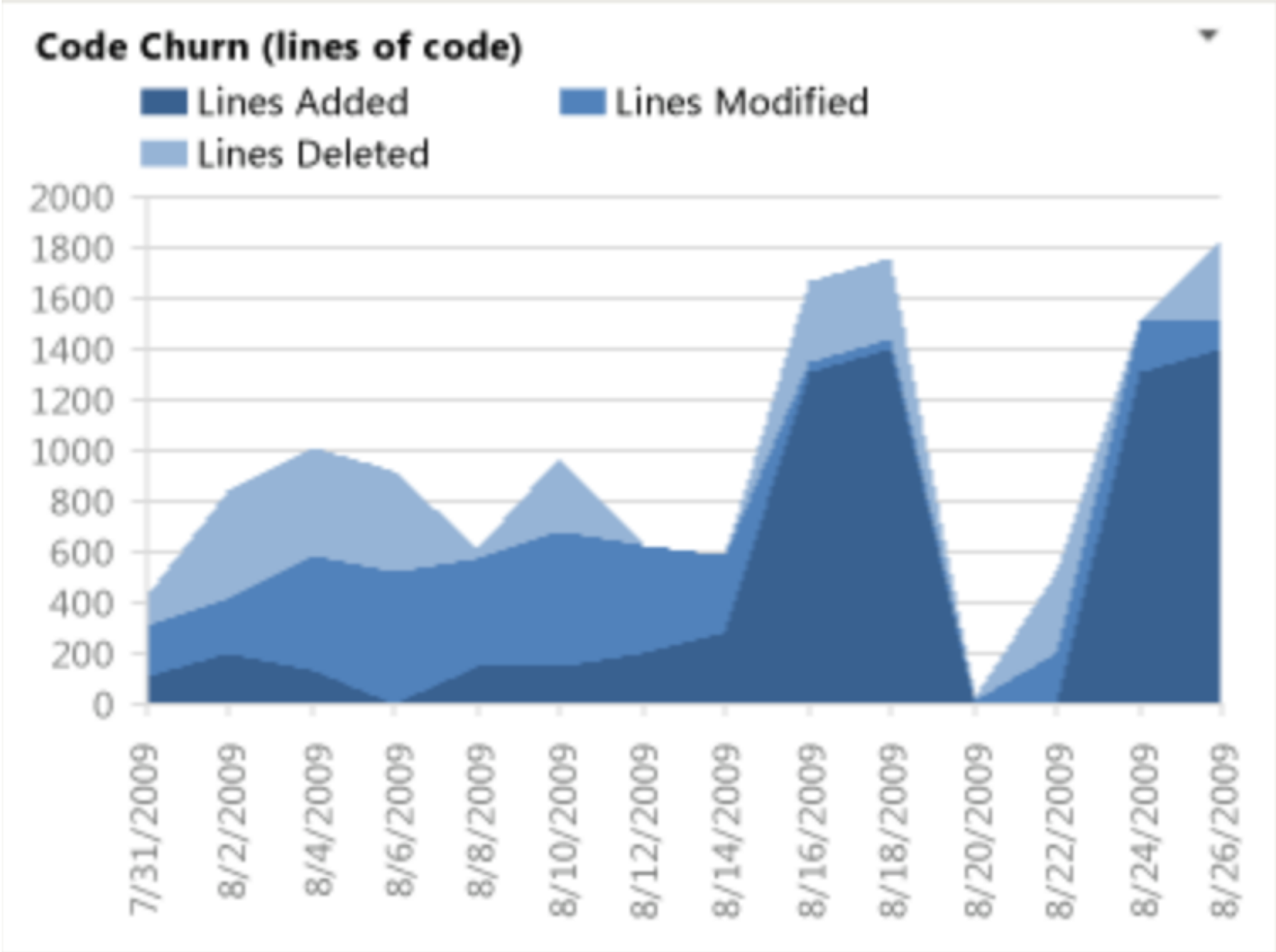
Software Metrics Characteristics I

- Process Metrics
  - Measure some aspect of the development process
    - project cost to date
    - Time spent so far on the project

} these change all the time

- Product Metrics
  - Measure some aspect of the software product.
    - analysis models
      - number of classes in an analysis class diagram.
    - test plans
    - program code

Example


Code Churn (lines of code)
Lines Added
Lines Modified
Lines Deleted
2000
1800
1600
1400
1200
1000
800
600
400
200
0
7/31/2009
8/2/2009
8/4/2009
8/6/2009
8/8/2009
8/10/2009
8/12/2009
8/14/2009
8/16/2009
8/18/2009
8/20/2009
8/22/2009
8/24/2009
8/26/2009

Software Metrics Characteristics II

- Result Metrics
  - measure outcomes
    - Current cost of a project (even though it will be modified tomorrow).
  - aka *Control Metrics*
    - used to determine how management control should be exercised.
      - measurement of the current level of progress in the project is used to decide whether action is necessary to bring the project back onto schedule.

Software Metrics Characteristics III

- Predictor Metrics
  - Quantify estimates for project resource requirements
    - Class size
      - a crude measure could be: simple count of attributes and operations
      - predictor because can be used to predict the time that it will take to produce code
  - Also a measure of some aspect of a software product that is used to predict *another* aspect of the product or project progress.
    - Predict that the system will be difficult to maintain
    - Predict very low levels of reuse

➢change the design to improve the system.

Are Software Metrics Worth Anything?

- Useful for prediction and resource estimation
  - Otherwise, is rather limited.
- Validity of predictor metrics is based on 3 assumptions
  - You can **measure something useful**
  - That measure **actually predicts something worthwhile**
  - This relationship is **real** and can be expressed in a model or a formula.
- Size metrics can be used to estimate the resource requirement for a project provided that appropriate historical data is available to derive and validate the relationship.

Metrics for Object-Oriented Development

- Ability of a package to absorb change is partly depends on ratio of abstract classes to all classes
  - 0 ⇒ package only has concrete classes and is difficult to change.
  - 1 ⇒ no concrete classes at all (easy to change!)
- Application size.
  - the number of use cases
  - the number of domain classes
  - ✖ multiplying factors that reflect the complexity of the user interface.
- Class size
  - number of attributes
  - number of operations
  - size of operations